Step 3 — A Triage Agent That Extracts a Structured Ticket From Free Text — Solution

> Back to this step · Hint 1 · Hint 2 · Solution

Story — Step 3: the chat can't file tickets yet. This step adds the first real agent — **Triage** — which turns a free-text complaint into a checked `SupportTicket` using structured output (the Doc04/Doc05 pattern). For now you trigger it by hand with `/extract`, so you can test the agent on its own before Step 4 automates the choice of when to use it. This is also where the app learns to survive every common failure — bad key, rate limit, too-long message, unusable model output — instead of crashing. **If not:** a support desk that can't produce structured tickets is just a chatbot, and one rate limit or bad reply would crash the whole session.

After this step the project runs as a chat app that can also file tickets. Files from Steps 1-2 that aren't shown here don't change.

Solution — a strict agent, every failure handled, sample data and tests

Story — `schemas.py`: this is the **contract** between the model and your code — a Pydantic `SupportTicket` model that says exactly which fields a ticket has and what types they are. Two choices do the heavy lifting: `str | None` makes "no name given" a *real, allowed* answer (so the model can honestly leave it blank), and `Literal[...]` restricts `category` and `urgency` to an exact set of values, so anything off-list is rejected at parse time. **If not:** a required `customer_name` would force the model to invent one, and a free-text category could come back as "Billing issue" — valid text that quietly breaks any `if category == "billing"`.

```
# src/schemas.py
# The shapes the model must answer in. SupportTicket: one support ticket.
from typing import Literal

from pydantic import BaseModel

class SupportTicket(BaseModel):
    customer_name: str | None
    issue_category: Literal["billing", "technical", "account_access", "other"]
```

```
    urgency: Literal["low", "medium", "high"]
    summary: str
```

Story — `prompts/triage.txt`: this is a strict prompt for a strict job. The schema *allows* `None` for a missing name, but a schema can't tell the model *when* to use it — the prompt does, by saying outright never to guess a name, and spelling out exactly when urgency counts as "high." Keeping these rules in the prompt (not the code) is what makes the model's behavior match the schema's intent. **If not:** the schema allows `None`, but nothing *tells* the model to use it, so it may still fill in a plausible-sounding name.

`src/prompts/triage.txt`

```
You are a support triage agent. Turn the customer's message into a
support ticket. Fill customer_name only if the customer states their
name - otherwise leave it null, never guess one. Use urgency 'high'
only if the customer says it is urgent or needed today.
```

Story — `agents/triage.py`: this is the **Triage agent**: its own prompt, no chat history (each complaint is judged on its own), and a single structured-output call that returns a `SupportTicket`. It checks the model's `refusal` field *before* reading `parsed`, and returns `None` on any unusable answer — so the caller always has exactly two clean cases to handle: a valid ticket, or nothing. **If not:** a refusal would come back as `parsed=None` with no explanation, and invalid output could slip through to code that trusts it.

```
# src/agents/triage.py
# The Triage agent: turns one complaint into a SupportTicket, or returns
# None when the model's answer can't be used.
from pydantic import ValidationError

from src.models.llm import MODEL
from src.schemas import SupportTicket
from src.utils.logger import get_logger
from src.utils.prompts import load_prompt

def triage_agent(client, message):
    logger = get_logger(__name__)
    messages = [
        {"role": "system", "content": load_prompt("triage")},
        {"role": "user", "content": message},
    ]
    try:
        completion = client.beta.chat.completions.parse(
            model=MODEL, messages=messages, response_format=SupportTicket
        )
    except ValidationError as e:
```

```
        logger.warning(f"Triage output did not match SupportTicket: {e}")
        return None

    reply = completion.choices[0].message
    if reply.refusal:
        logger.warning(f"Triage agent refused: {reply.refusal}")
        return None
    return reply.parsed
```

Also create an empty `src/agents/__init__.py`.

Story — `services/desk_service.py`: this is the home for the app's **business flow** — what happens to a message, separate from reading the keyboard. In this step it holds just one function, `print_ticket()`, which displays a ticket readably. It's created now, even for one function, because Step 4 adds the whole router → agent flow to *this same file* — so putting it here now means nothing has to move later. (Also create an empty `src/services/__init__.py`.) **If not:** `print_ticket()` would sit in `main.py` now and have to be moved in Step 4, churning both files.

```
# src/services/desk_service.py
# The app's flow, apart from the keyboard. print_ticket() shows a ticket.

def print_ticket(ticket):
    name = ticket.customer_name
    if name is None:
        name = "(not given)"
    print("Ticket created:")
    print(f"  customer: {name}")
    print(f"  category: {ticket.issue_category}")
    print(f"  urgency:  {ticket.urgency}")
    print(f"  summary:  {ticket.summary}")
```

Changed from Step 2: the startup and the chat branch are the same. What's new: 3 imports, the `/extract` branch that calls `print_ticket()`, and the `try` / `except` blocks around the loop body.

Story — `main.py`: this is Step 2's loop plus the `/extract` command and one `except` clause per failure mode. Each failure gets the exact reaction Doc04's error table prescribes: exit on a bad key, ask the user to wait on a rate limit, and drop a too-long message. (The fourth failure — bad structured output — is already handled inside `triage_agent()`, which returns `None`.) Handling each error where the user can see a clean message keeps one bad call from ending the session. **If not:** any one of these would end the session with a traceback, and the user would lose the whole conversation.

```
# main.py
# The file you run: python main.py
```

```
# The chat loop, plus /extract <text> to file a ticket, and one clear
# message for each OpenAI error (bad key, rate limit, too long).
import sys

# new in Step 3: openai (for its error classes), triage_agent, print_ticket
import openai

from src.agents.triage import triage_agent
from src.config import load_config
from src.exceptions import MissingConfigError
from src.models.llm import (
    check_api_key_at_startup,
    create_client,
    stream_message,
)
from src.services.desk_service import print_ticket
from src.utils.history import trim_history
from src.utils.logger import get_logger
from src.utils.prompts import load_prompt

def main():
    try:
        config = load_config()
    except MissingConfigError as e:
        print(f"Setup problem: {e}")
        sys.exit(1)

    logger = get_logger(__name__)
    client = create_client(config)
    if not check_api_key_at_startup(client):
        logger.error("Authentication failed — check OPENAI_API_KEY.")
        print("Login failed — check OPENAI_API_KEY in your .env file.")
        sys.exit(1)

    history = [{"role": "system", "content": load_prompt("concierge")}]
    print("SupportDesk AI — type a message, or /quit to exit.")
    print("Type /extract <complaint> to file a ticket.")

    while True:
        message = input("You: ").strip()
        if message == "/quit":
            break
        if message == "":
            continue

        # new in Step 3: try / except around the loop body, and /extract
        try:
```

```
            if message.startswith("/extract "):
                text = message.removeprefix("/extract ")
                ticket = triage_agent(client, text)
                if ticket is None:
                    print("Sorry, I couldn't turn that into a ticket.")
                else:
                    print_ticket(ticket)
            else:
                trim_history(history, 3000)
                print("Assistant: ", end="", flush=True)
                stream_message(client, history, message)
        except openai.AuthenticationError:
            logger.error("API key stopped working mid-session.")
            print("Login failed — check OPENAI_API_KEY in your .env file.")
            sys.exit(1)
        except openai.RateLimitError:
            logger.warning("Rate limited after the client's retries.")
            print("We're busy right now — wait a moment and try again.")
        except openai.BadRequestError as e:
            logger.warning(f"Request rejected (likely too long): {e}")
            if history[-1]["role"] == "user":
                history.pop()
            print("That message was too long to handle — please shorten it.")

if __name__ == "__main__":
    main()
```

Why these `except` blocks sit in `main.py`: this is the only place that knows how to talk to the *user* (print a message, or exit). The agents just raise or return `None`.

Story — `data/sample/messages.json`: this stores the 3 test complaints from the README once, in one file. Both the sample-runner script and the integration test read from it, so there's a single source of truth for the examples. **If not:** the script and the test would each keep their own copy of the complaints, and the two copies would quietly drift apart over time.

`data/sample/messages.json`

```
{
  "triage": [
    "Hi, I'm Sarah Khan. I was charged twice this month. Please fix it today.",
    "can't log into my account, tried resetting password twice, still nothing",
    "just wondering what your refund policy is, no rush"
  ]
}
```

(The first message is wrapped onto 2 lines here only to fit the page — in your file keep it on **one** line, because a JSON string can't contain a line break.)

Story — `scripts/triage_samples.py`: this runs the Triage agent on every sample complaint and prints each resulting ticket, for *you* to eyeball — for example right after rewording `triage.txt`, to see whether the change helped or hurt. It's a judgement tool, not a pass/fail test. **If not:** checking a prompt change would mean typing the 3 complaints in by hand every single time.

```
# scripts/triage_samples.py
# Runs the Triage agent on every sample complaint and prints each ticket.
# Run from the repo root: python -m scripts.triage_samples
# Makes real API calls — needs a valid OPENAI_API_KEY in .env.
import json

from src.agents.triage import triage_agent
from src.config import load_config
from src.models.llm import create_client

SAMPLES_PATH = "data/sample/messages.json"

def main():
    config = load_config()
    client = create_client(config)
    with open(SAMPLES_PATH) as samples_file:
        samples = json.loads(samples_file.read())

    for message in samples["triage"]:
        print(f"\nMessage: {message}")
        ticket = triage_agent(client, message)
        if ticket is None:
            print("No ticket — the Triage agent couldn't extract one.")
        else:
            print(f"Ticket:  {ticket}")

if __name__ == "__main__":
    main()
```

Story — `tests/unit/test_schemas.py`: this checks the schema's two key promises without any model or network — free and instant: that a ticket is accepted with a missing name (`None`), and that an unknown `urgency` value is rejected. It pins the schema's behavior in place so a future change can't quietly loosen it. **If not:** someone could change `customer_name` to a plain `str` and nothing would warn them until the model started inventing names in production.

```
# tests/unit/test_schemas.py
# Checks the Pydantic schemas accept good data and reject bad data.
```

```
# Each case prints OK or WRONG. No API calls, no API key needed.
# Run from the repo root: python -m tests.unit.test_schemas
from pydantic import ValidationError

from src.schemas import SupportTicket

def test_ticket_without_a_name():
    ticket = SupportTicket(
        customer_name=None,
        issue_category="account_access",
        urgency="medium",
        summary="Customer can't log in after two password resets.",
    )
    result = "OK"
    if ticket.customer_name is not None:
        result = "WRONG"
    print(f"case 1 (no name allowed)  -> {result}")

def test_ticket_rejects_unknown_urgency():
    result = "WRONG"
    try:
        SupportTicket(
            customer_name="Sarah Khan",
            issue_category="billing",
            urgency="urgent",
            summary="Charged twice.",
        )
    except ValidationError:
        result = "OK"
    print(f"case 2 (bad urgency)      -> {result}")

if __name__ == "__main__":
    test_ticket_without_a_name()
    test_ticket_rejects_unknown_urgency()
```

Changed from Step 2: `tests/integration/test_live_agents.py` gets cases 4 and 5 — `load_samples()`, two test functions, their imports, and three new lines at the bottom.

Story — `tests/integration/test_live_agents.py`: the Triage agent is this step's feature, so it gets a real check against the live model: each of the 3 sample complaints must come back as a valid ticket, and the one complaint with no name must *not* get an invented one. This case is added to the same growing integration file. **If not:** a prompt change could start inventing customer names, and nothing would catch it until a user noticed.

```
# tests/integration/test_live_agents.py
# Checks the app against the real OpenAI API — each step adds cases.
```

```
# Each case prints OK or WRONG. Needs a valid OPENAI_API_KEY in .env.
# Run from the repo root: python -m tests.integration.test_live_agents
# new in Step 3
import json

from openai import OpenAI

# new in Step 3
from src.agents.triage import triage_agent
from src.config import load_config
from src.models.llm import (
    check_api_key_at_startup,
    create_client,
    send_message,
)

# new in Step 3
SAMPLES_PATH = "data/sample/messages.json"

def test_first_reply(client):
    reply = send_message(client, [], "What's 2+2?")
    result = "OK"
    if "4" not in reply:
        result = "WRONG"
    print(f"case 1 (first reply) -> {result}: {reply}")

def test_bad_key():
    bad_client = OpenAI(api_key="sk-fake-key-123")
    result = "OK"
    if check_api_key_at_startup(bad_client):
        result = "WRONG"
    print(f"case 2 (bad key)     -> {result}")

def test_memory(client):
    history = [{"role": "system", "content": "You are a helpful assistant."}]
    send_message(client, history, "My order number is 4471.")
    send_message(client, history, "Thanks, that's all for now.")
    reply = send_message(client, history, "What was my order number?")
    result = "OK"
    if "4471" not in reply:
        result = "WRONG"
    print(f"case 3 (memory)      -> {result}: {reply}")

# new in Step 3
def load_samples():
    with open(SAMPLES_PATH) as samples_file:
        return json.loads(samples_file.read())
```

```
# new in Step 3
def test_every_sample_gets_a_ticket(client, samples):
    for message in samples["triage"]:
        ticket = triage_agent(client, message)
        if ticket is None:
            print("case 4 (ticket)      -> WRONG: no ticket")
        else:
            print("case 4 (ticket)      -> OK:", ticket.issue_category)

# new in Step 3
def test_no_invented_name(client, samples):
    ticket = triage_agent(client, samples["triage"][1])
    result = "OK"
    if ticket is None or ticket.customer_name is not None:
        result = "WRONG"
    print(f"case 5 (no name)     -> {result}")

if __name__ == "__main__":
    config = load_config()
    client = create_client(config)
    # new in Step 3
    samples = load_samples()
    test_first_reply(client)
    test_bad_key()
    test_memory(client)
    # new in Step 3
    test_every_sample_gets_a_ticket(client, samples)
    test_no_invented_name(client, samples)
```

Run everything:

```
python -m tests.unit.test_config
python -m tests.unit.test_history
python -m tests.unit.test_schemas
python -m tests.integration.test_live_agents
python -m scripts.triage_samples
python main.py
```

Expected output of `test_schemas`:

```
case 1 (no name allowed)  -> OK
case 2 (bad urgency)      -> OK
```

Expected output of `test_live_agents` (the model's wording varies):

```
case 1 (first reply) -> OK: 2 + 2 equals 4.
case 2 (bad key)     -> OK
case 3 (memory)      -> OK: Your order number was 4471.
case 4 (ticket)      -> OK: billing
case 4 (ticket)      -> OK: account_access
case 4 (ticket)      -> OK: other
case 5 (no name)     -> OK
```

In the app, `/extract can't log into my account, tried resetting password twice` prints a ticket with `customer: (not given)`.